Velo

Banco de dados

Diagrama de relacionamentos e arquitetura

Feito por Erik L. Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 3

- Bancos suportados 3
- As 13 tabelas 3

Diagrama de relacionamentos 4

- Como ler o diagrama 5
- Ligações 5

Regras e convenções 10

- Convenções do modelo 10
- Tipos em cada banco 10
- Fluxo de situações do pedido 11

Tabelas 12

- vl_user 12
- vl_session 12
- vl_address 13
- vl_category 14
- vl_product 14
- vl_cart 15
- vl_cart_item 15
- vl_coupon 16
- vl_order 16
- vl_order_item 17
- vl_order_status_history 18
- vl_payment 18
- vl_inventory_movement 19

Como parear o banco 20

- Sem configurar nada (H2) 20
- 1. Subir o banco 20
- 2. Criar o banco e o usuário (sem Docker) 20
- 3. Apontar a Velo para o banco 21
- Problemas comuns 22

Evolução do esquema 23

Capítulo 1

Visão geral

Este documento descreve o modelo de dados que a Velo usa e que qualquer banco escolhido precisa seguir: as 13 tabelas, seus campos, tipos, chaves e relacionamentos, além de como ligar a aplicação a cada banco.

Bancos suportados

O mesmo modelo roda em seis bancos. As tabelas são criadas sozinhas na primeira execução pelo Flyway, a partir do script `V1__schema.sql` do banco escolhido, em `backend/src/main/resources/db/migration/`.

Banco	Versão mínima	Perfil	Pasta do script
H2 (arquivo local)	2.2	nenhum (padrão)	`h2/`
PostgreSQL	13	`postgresql`	`postgresql/`
MySQL	8.0	`mysql`	`mysql/`
MariaDB	10.6	`mariadb`	`mariadb/`
SQL Server	2019	`sqlserver`	`sqlserver/`
Oracle	19c	`oracle`	`oracle/`

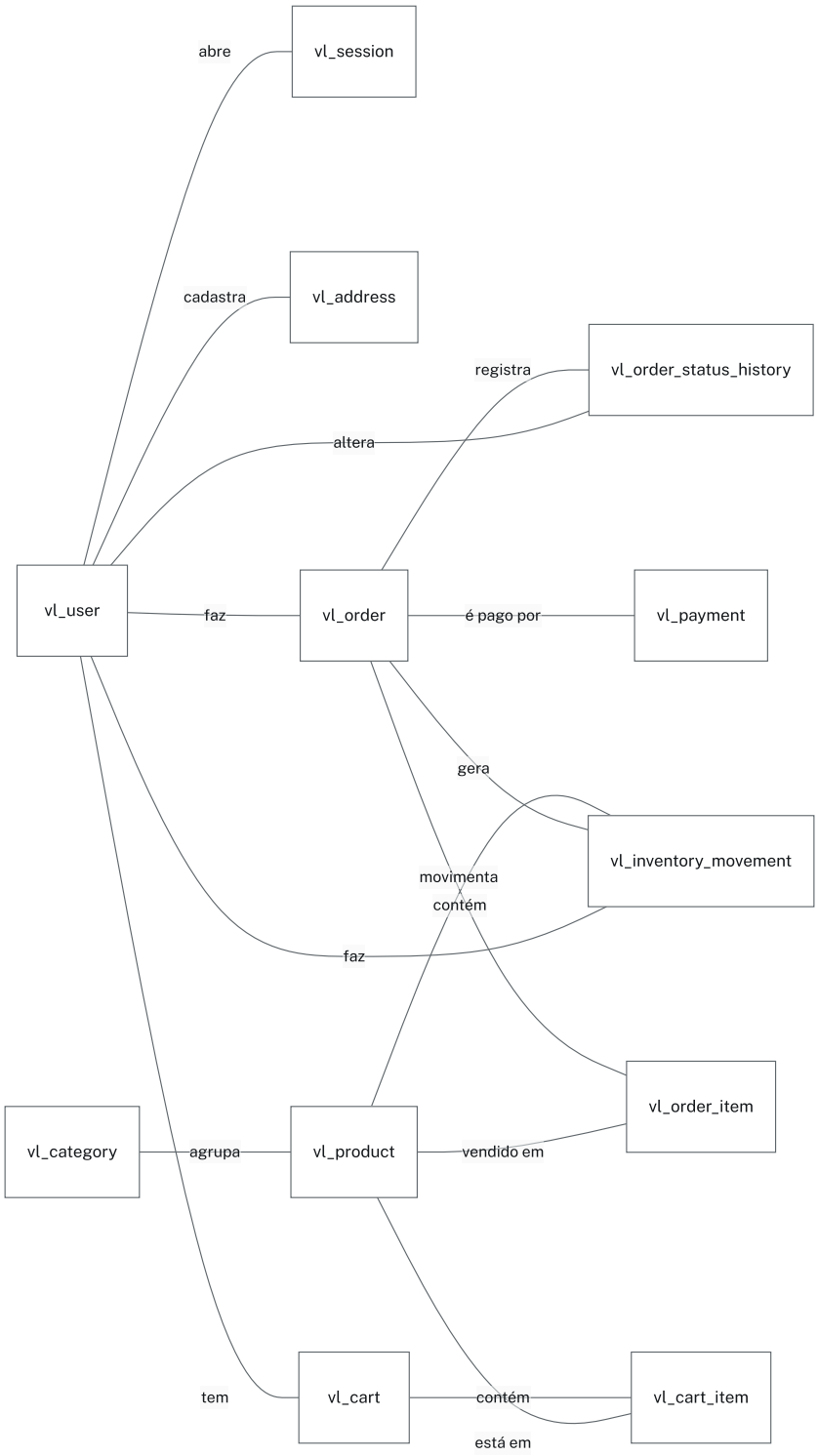
As 13 tabelas

Grupo	Tabela	O que guarda
Pessoas e acesso	`vl_user`	Pessoas que acessam a Velo: clientes, equipe e administradores.
	`vl_session`	Sessões de login. O token só existe na memória do navegador; aqui fica apenas o hash.
	`vl_address`	Endereços de entrega dos clientes.
Catálogo e estoque	`vl_category`	Categorias do catálogo.
	`vl_product`	Produtos à venda, com preço e estoque.
	`vl_inventory_movement`	Histórico de entradas e saídas de estoque.
Carrinho e cupons	`vl_cart`	Carrinho de compras, um por cliente.
	`vl_cart_item`	Itens do carrinho.
	`vl_coupon`	Cupons de desconto.
Pedidos e pagamentos	`vl_order`	Pedidos. O endereço é copiado no momento da compra.
	`vl_order_item`	Itens do pedido, com nome e preço copiados na hora da compra.
	`vl_order_status_history`	Linha do tempo de cada pedido.
	`vl_payment`	Pagamentos dos pedidos. Número de cartão nunca é gravado.

Capítulo 2

Diagrama de relacionamentos

Primeiro a visão das ligações entre as tabelas; em seguida o diagrama completo, com todos os campos e chaves.



Ligações entre as tabelas

Como ler o diagrama

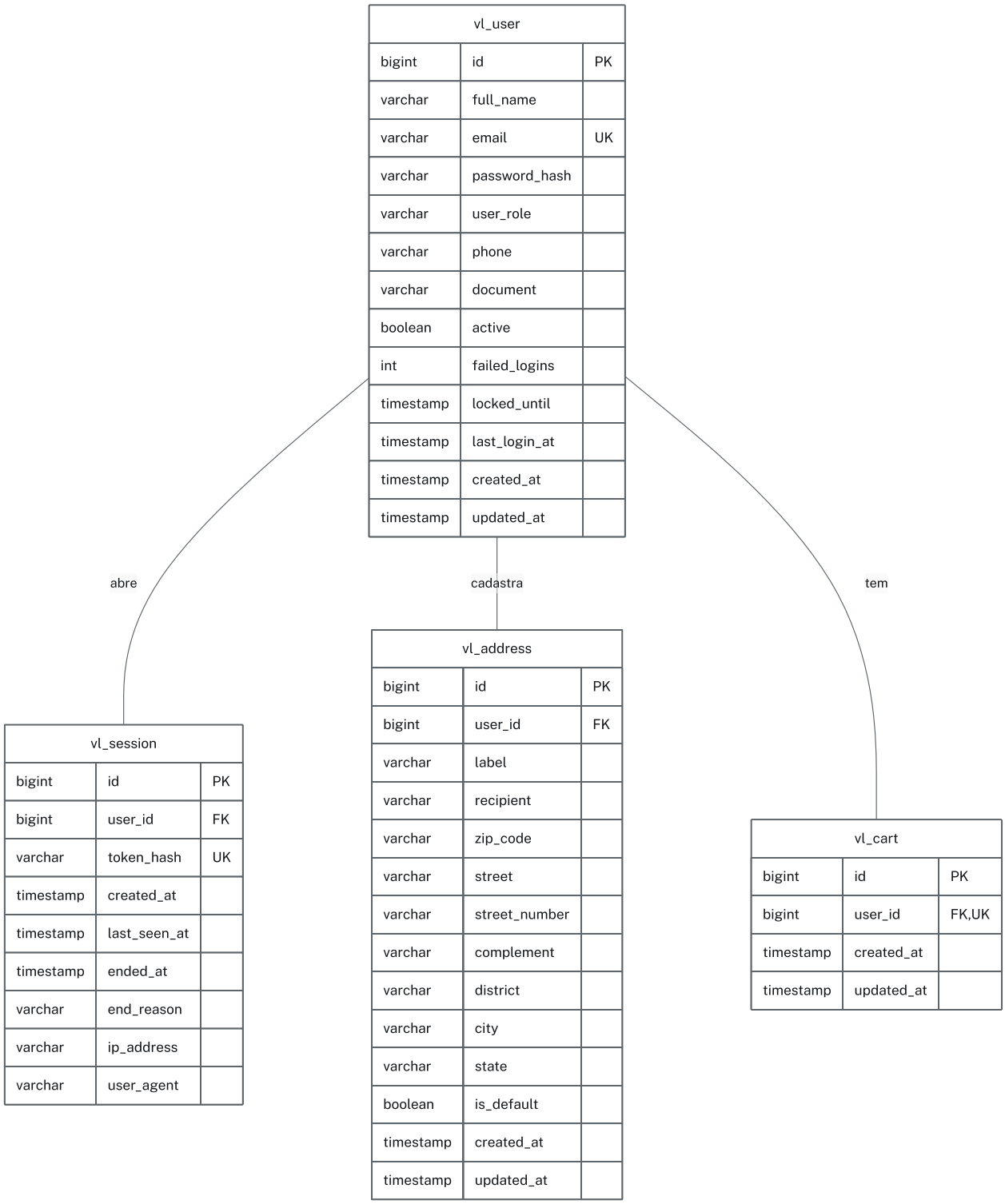
Símbolo na ponta da linha	Significado
`||` (dois traços)	exatamente um
`|o` (traço e círculo)	zero ou um
`}|` (pé de galinha e traço)	um ou mais
`}o` (pé de galinha e círculo)	zero ou mais
PK	chave primária
FK	chave estrangeira, aponta para o id de outra tabela
UK	valor único na tabela

Ligações

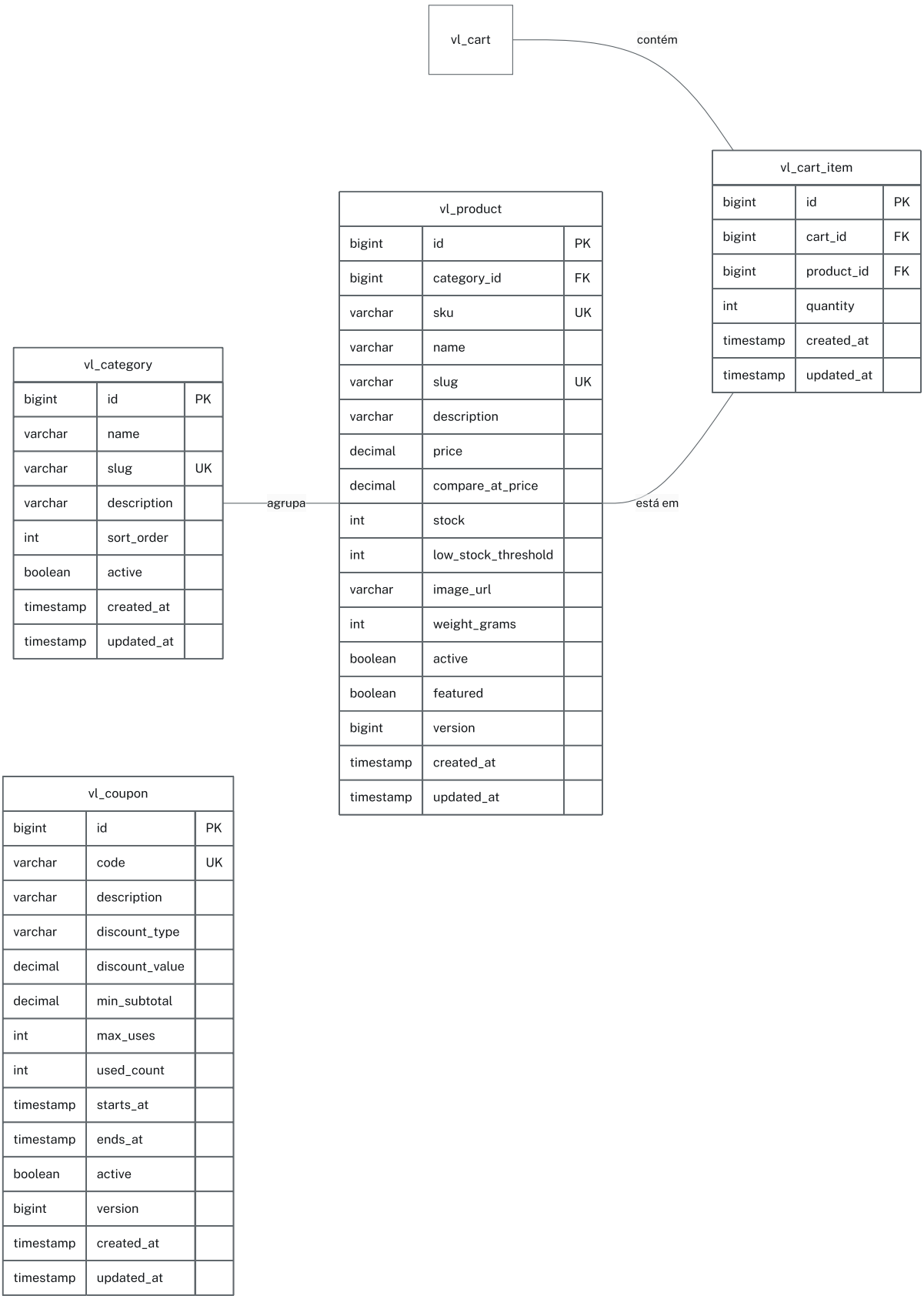
- Um **usuário** tem várias **sessões**, vários **endereços**, no máximo um **carrinho** e vários **pedidos**.
- Um **carrinho** tem vários **itens**, cada um apontando para um **produto**; o mesmo produto aparece uma única vez por carrinho.
- Um **produto** pertence a no máximo uma **categoria** e tem um histórico de **movimentos de estoque**.
- Um **pedido** tem um ou mais **itens**, uma **linha do tempo** de situações e um ou mais **pagamentos** (cada nova tentativa cria um pagamento).
- Um **movimento de estoque** pode estar ligado ao pedido que o gerou e ao usuário que o fez.
- **Cupons** se ligam ao pedido pelo código (`vl_order.coupon_code`), sem chave estrangeira, para o pedido continuar íntegro mesmo se o cupom for excluído.

Diagrama completo

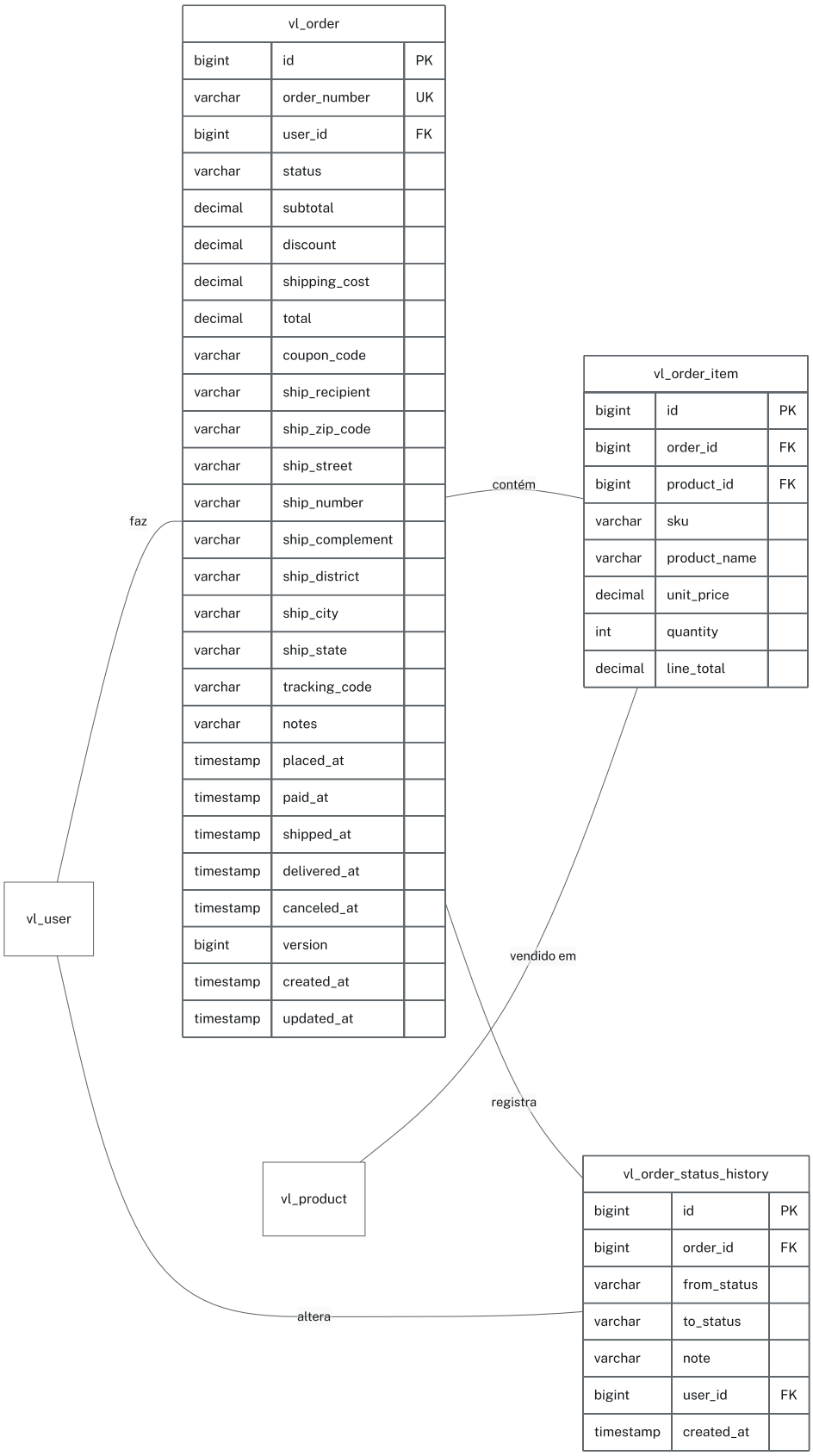
Todas as tabelas com tipos, campos e chaves, divididas em quatro partes para caber na página; as tabelas marcadas como referência têm os campos detalhados na parte em que aparecem completas. Use o zoom do leitor de PDF para ver os detalhes.



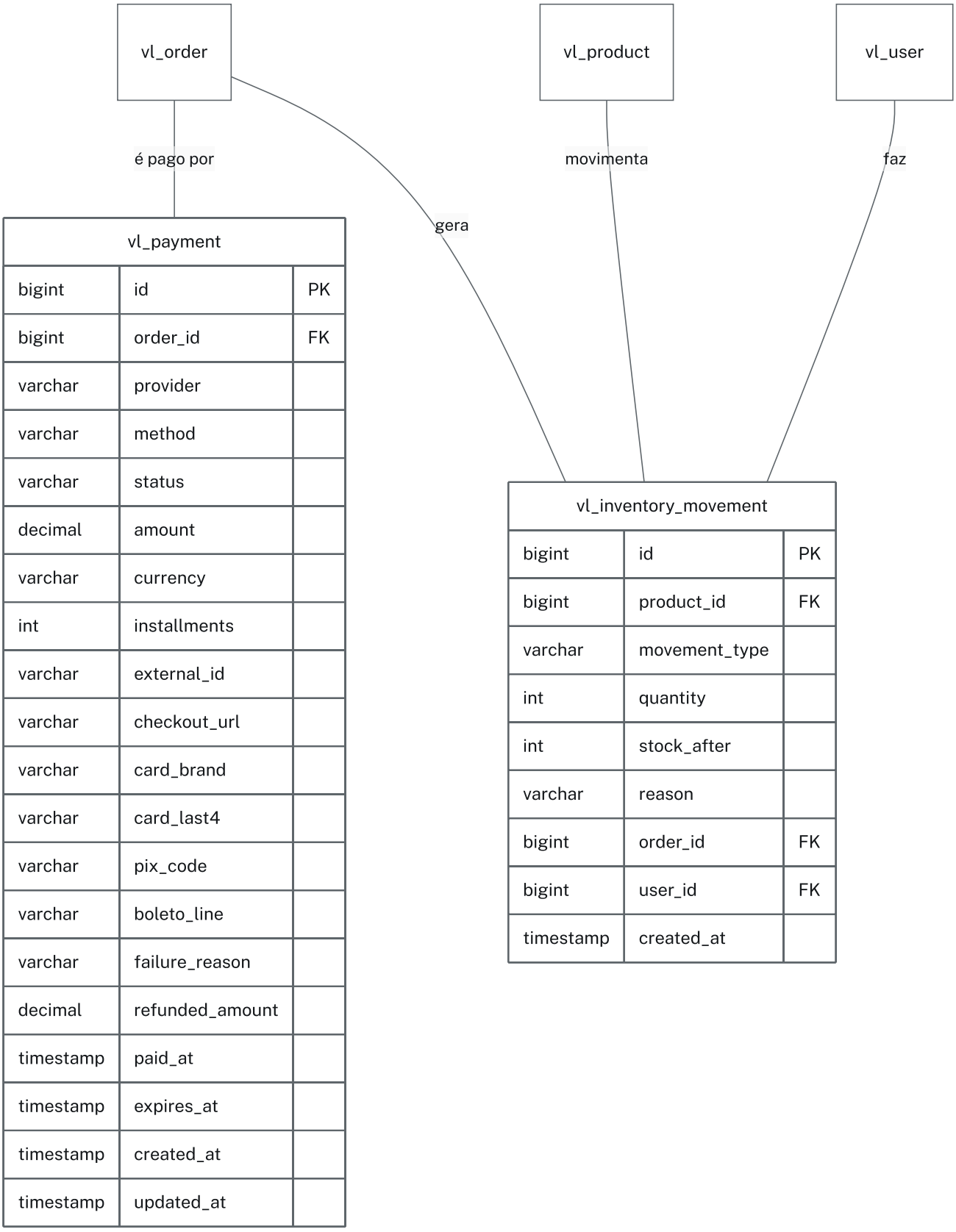
Parte 1 de 4: pessoas e acesso (`vl_user`, `vl_session`, `vl_address`, `vl_cart`)



Parte 2 de 4: catálogo, itens do carrinho e cupons (`vl_category`, `vl_product`, `vl_cart_item`, `vl_coupon`). `vl_cart` aparece só como referência.



Parte 3 de 4: pedidos (`vl_order`, `vl_order_item`, `vl_order_status_history`). `vl_user` e `vl_product` aparecem só como referência.



vl_payment		
bigint	id	PK
bigint	order_id	FK
varchar	provider	
varchar	method	
varchar	status	
decimal	amount	
varchar	currency	
int	installments	
varchar	external_id	
varchar	checkout_url	
varchar	card_brand	
varchar	card_last4	
varchar	pix_code	
varchar	boleto_line	
varchar	failure_reason	
decimal	refunded_amount	
timestamp	paid_at	
timestamp	expires_at	
timestamp	created_at	
timestamp	updated_at	

vl_inventory_movement		
bigint	id	PK
bigint	product_id	FK
varchar	movement_type	
int	quantity	
int	stock_after	
varchar	reason	
bigint	order_id	FK
bigint	user_id	FK
timestamp	created_at	

Parte 4 de 4: pagamentos e estoque (`vl_payment`, `vl_inventory_movement`). `vl_order`, `vl_product` e `vl_user` aparecem só como referência.

Capítulo 3

Regras e convenções

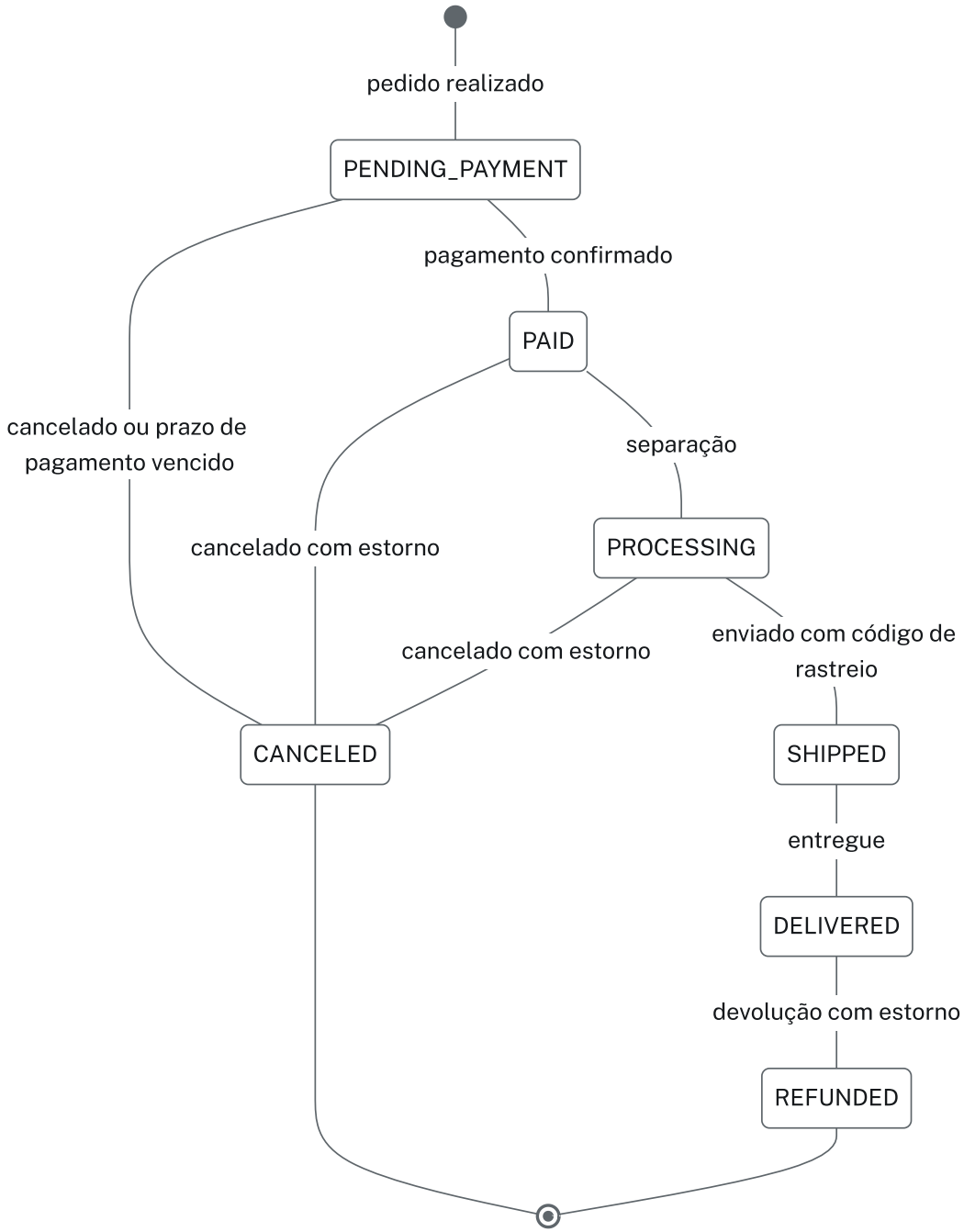
Convenções do modelo

- Todas as tabelas começam com `vl_` e a chave primária se chama `id` (inteiro de 64 bits, autoincremento).
- Datas e horas ficam em **UTC**; a interface converte para o horário de Brasília.
- Valores em dinheiro usam `decimal(12,2)` e a moeda é sempre BRL.
- Situações e tipos são gravados como texto em maiúsculas (por exemplo `PENDING_PAYMENT`), legíveis em qualquer banco.
- Pedidos copiam o endereço e o nome e preço de cada item no momento da compra; alterar um produto depois não muda pedidos antigos.
- Senhas são guardadas só como hash BCrypt, sessões só com o SHA-256 do token e cartões só com a bandeira e os 4 últimos dígitos.
- A coluna `version` em produto, cupom e pedido impede que duas pessoas sobrescrevam a mesma alteração.
- Os índices são criados antes das chaves estrangeiras, para o mesmo script funcionar em todos os bancos.

Tipos em cada banco

Tipo no modelo	H2 / PostgreSQL	MySQL / MariaDB	SQL Server	Oracle
bigint, autoincremento	`BIGINT GENERATED BY DEFAULT AS IDENTITY`	`BIGINT AUTO_INCREMENT`	`BIGINT IDENTITY(1,1)`	`NUMBER(19) GENERATED BY DEFAULT ON NULL AS IDENTITY`
bigint	`BIGINT`	`BIGINT`	`BIGINT`	`NUMBER(19)`
inteiro	`INTEGER`	`INT`	`INT`	`NUMBER(10)`
texto(n)	`VARCHAR(n)`	`VARCHAR(n)` utf8mb4	`NVARCHAR(n)`	`VARCHAR2(n CHAR)`
decimal(12,2)	`NUMERIC(12,2)`	`DECIMAL(12,2)`	`DECIMAL(12,2)`	`NUMBER(12,2)`
booleano	`BOOLEAN`	`BOOLEAN`	`BIT`	`NUMBER(1)`
data e hora	`TIMESTAMP(6)`	`DATETIME(6)`	`DATETIME2(6)`	`TIMESTAMP(6)`

Fluxo de situações do pedido



Situações gravadas em `vl_order.status` e registradas em `vl_order_status_history`

O estoque sai no momento da compra (movimento `SALE`) e volta quando o pedido é cancelado ou estornado (movimento `RETURN`). Pedidos não pagos em 60 minutos são cancelados automaticamente.

Capítulo 4

Tabelas

Cada tabela com todos os campos. "Obrigatório" indica NOT NULL.

vl_user

Pessoas que acessam a Velo: clientes, equipe e administradores.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
full_name	texto(120)	sim		Nome completo
email	texto(160)	sim	Único	E-mail de login, único
password_hash	texto(100)	sim		Hash BCrypt da senha
user_role	texto(20)	sim		Perfil de acesso: ADMIN, STAFF, CUSTOMER
phone	texto(30)	não		Telefone
document	texto(20)	não		CPF ou CNPJ
active	booleano	sim		Conta ativa
failed_logins	inteiro	sim		Tentativas de login erradas seguidas
locked_until	data e hora	não		Bloqueio temporário após várias tentativas erradas
last_login_at	data e hora	não		Último login
created_at	data e hora	sim		Data de criação (UTC)
updated_at	data e hora	sim		Última alteração (UTC)

Índices: user_role.

vl_session

Sessões de login. O token só existe na memória do navegador; aqui fica apenas o hash.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
user_id	bigint	sim	FK: vl_user.id	Dono da sessão
token_hash	texto(64)	sim	Único	SHA-256 do token entregue ao navegador
created_at	data e hora	sim		Início da sessão; limite de 30 minutos a partir daqui
last_seen_at	data e hora	sim		Última atividade; limite de 20 minutos parado
ended_at	data e hora	não		Quando a sessão foi encerrada

Coluna	Tipo	Obrig.	Chave	Descrição
`end_reason`	texto(20)	não		Motivo do encerramento: LOGOUT, IDLE, EXPIRED, REVOKED
`ip_address`	texto(45)	não		IP de origem
`user_agent`	texto(255)	não		Navegador

Índices: `user_id`.

`vl_address`

Endereços de entrega dos clientes.

Coluna	Tipo	Obrig.	Chave	Descrição
`id`	bigint, autoincremento	sim	PK	
`user_id`	bigint	sim	FK: vl_user.id	Cliente
`label`	texto(40)	sim		Apelido (Casa, Trabalho)
`recipient`	texto(120)	sim		Quem recebe
`zip_code`	texto(12)	sim		CEP
`street`	texto(160)	sim		Logradouro
`street_number`	texto(20)	sim		Número
`complement`	texto(80)	não		Complemento
`district`	texto(80)	sim		Bairro
`city`	texto(80)	sim		Cidade
`state`	texto(2)	sim		UF
`is_default`	booleano	sim		Endereço principal
`created_at`	data e hora	sim		Data de criação (UTC)
`updated_at`	data e hora	sim		Última alteração (UTC)

Índices: `user_id`.

vl_category

Categorias do catálogo.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
name	texto(80)	sim		Nome
slug	texto(90)	sim	Único	Identificador na URL, único
description	texto(400)	não		Descrição
sort_order	inteiro	sim		Ordem de exibição
active	booleano	sim		Visível na loja
created_at	data e hora	sim		Data de criação (UTC)
updated_at	data e hora	sim		Última alteração (UTC)

vl_product

Produtos à venda, com preço e estoque.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
category_id	bigint	não	FK: vl_category.id	Categoria
sku	texto(40)	sim	Único	Código do produto, único
name	texto(140)	sim		Nome
slug	texto(160)	sim	Único	Identificador na URL, único
description	texto(2000)	não		Descrição
price	decimal(12,2)	sim		Preço de venda
compare_at_price	decimal(12,2)	não		Preço antigo, para mostrar desconto
stock	inteiro	sim		Quantidade disponível
low_stock_threshold	inteiro	sim		Alerta de estoque baixo
image_url	texto(400)	não		Imagem principal
weight_grams	inteiro	sim		Peso para frete
active	booleano	sim		Visível na loja
featured	booleano	sim		Destaque na vitrine
version	bigint	não		Controle de concorrência
created_at	data e hora	sim		Data de criação (UTC)
updated_at	data e hora	sim		Última alteração (UTC)

Índices: `category_id`, `active`.

vl_cart

Carrinho de compras, um por cliente.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
user_id	bigint	sim	FK: vl_user.id Único	Cliente
created_at	data e hora	sim		Data de criação (UTC)
updated_at	data e hora	sim		Última alteração (UTC)

vl_cart_item

Itens do carrinho.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
cart_id	bigint	sim	FK: vl_cart.id	Carrinho
product_id	bigint	sim	FK: vl_product.id	Produto
quantity	inteiro	sim		Quantidade
created_at	data e hora	sim		Data de criação (UTC)
updated_at	data e hora	sim		Última alteração (UTC)

Unicidade composta: (cart_id, product_id).

vl_coupon

Cupons de desconto.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
code	texto(30)	sim	Único	Código digitado pelo cliente, único
description	texto(200)	não		Descrição
discount_type	texto(20)	sim		Tipo de desconto: PERCENT, FIXED
discount_value	decimal(12,2)	sim		Percentual ou valor
min_subtotal	decimal(12,2)	não		Subtotal mínimo
max_uses	inteiro	não		Limite de usos
used_count	inteiro	sim		Usos até agora
starts_at	data e hora	não		Início da validade
ends_at	data e hora	não		Fim da validade
active	booleano	sim		Ativo
version	bigint	não		Controle de concorrência
created_at	data e hora	sim		Data de criação (UTC)
updated_at	data e hora	sim		Última alteração (UTC)

vl_order

Pedidos. O endereço é copiado no momento da compra.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
order_number	texto(20)	sim	Único	Número exibido ao cliente, único
user_id	bigint	sim	FK: vl_user.id	Cliente
status	texto(20)	sim		Situação: PENDING_PAYMENT, PAID, PROCESSING, SHIPPED, DELIVERED, CANCELED, REFUNDED
subtotal	decimal(12,2)	sim		Soma dos itens
discount	decimal(12,2)	sim		Desconto do cupom
shipping_cost	decimal(12,2)	sim		Frete
total	decimal(12,2)	sim		Total cobrado
coupon_code	texto(30)	não		Cupom usado
ship_recipient	texto(120)	sim		Destinatário
ship_zip_code	texto(12)	sim		CEP
ship_street	texto(160)	sim		Logradouro
ship_number	texto(20)	sim		Número

Coluna	Tipo	Obrig.	Chave	Descrição
`ship_complement`	texto(80)	não		Complemento
`ship_district`	texto(80)	sim		Bairro
`ship_city`	texto(80)	sim		Cidade
`ship_state`	texto(2)	sim		UF
`tracking_code`	texto(40)	não		Código de rastreio
`notes`	texto(500)	não		Observações
`placed_at`	data e hora	sim		Data da compra
`paid_at`	data e hora	não		Pagamento confirmado
`shipped_at`	data e hora	não		Enviado
`delivered_at`	data e hora	não		Entregue
`canceled_at`	data e hora	não		Cancelado
`version`	bigint	não		Controle de concorrência
`created_at`	data e hora	sim		Data de criação (UTC)
`updated_at`	data e hora	sim		Última alteração (UTC)

Índices: `user_id`, `status`, `placed_at`.

`vl_order_item`

Itens do pedido, com nome e preço copiados na hora da compra.

Coluna	Tipo	Obrig.	Chave	Descrição
`id`	bigint, autoincremento	sim	PK	
`order_id`	bigint	sim	FK: vl_order.id	Pedido
`product_id`	bigint	não	FK: vl_product.id	Produto (pode ficar vazio se o produto for excluído)
`sku`	texto(40)	sim		Código na hora da compra
`product_name`	texto(140)	sim		Nome na hora da compra
`unit_price`	decimal(12,2)	sim		Preço unitário
`quantity`	inteiro	sim		Quantidade
`line_total`	decimal(12,2)	sim		Preço x quantidade

Índices: `order_id`, `product_id`.

vl_order_status_history

Linha do tempo de cada pedido.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
order_id	bigint	sim	FK: vl_order.id	Pedido
from_status	texto(20)	não		Situação anterior
to_status	texto(20)	sim		Nova situação
note	texto(300)	não		Observação
user_id	bigint	não	FK: vl_user.id	Quem alterou (vazio = sistema)
created_at	data e hora	sim		Quando

Índices: order_id.

vl_payment

Pagamentos dos pedidos. Número de cartão nunca é gravado.

Coluna	Tipo	Obrig.	Chave	Descrição
id	bigint, autoincremento	sim	PK	
order_id	bigint	sim	FK: vl_order.id	Pedido
provider	texto(20)	sim		Provedor: SIMULATED, STRIPE
method	texto(20)	sim		Forma de pagamento: CARD, PIX, BOLETO, CHECKOUT
status	texto(20)	sim		Situação: PENDING, PAID, FAILED, CANCELED, REFUNDED
amount	decimal(12,2)	sim		Valor
currency	texto(3)	sim		Moeda (BRL)
installments	inteiro	sim		Parcelas
external_id	texto(120)	não		Identificador no provedor
checkout_url	texto(500)	não		Página de pagamento do provedor
card_brand	texto(20)	não		Bandeira
card_last4	texto(4)	não		Últimos 4 dígitos
pix_code	texto(300)	não		Código PIX copia e cola
boleto_line	texto(60)	não		Linha digitável do boleto
failure_reason	texto(200)	não		Motivo da recusa
refunded_amount	decimal(12,2)	sim		Valor estornado
paid_at	data e hora	não		Confirmação
expires_at	data e hora	não		Validade do PIX ou boleto

Coluna	Tipo	Obrig.	Chave	Descrição
`created_at`	data e hora	sim		Data de criação (UTC)
`updated_at`	data e hora	sim		Última alteração (UTC)

Índices: `order_id`, `external_id`.

`vl_inventory_movement`

Histórico de entradas e saídas de estoque.

Coluna	Tipo	Obrig.	Chave	Descrição
`id`	bigint, autoincremento	sim	PK	
`product_id`	bigint	sim	FK: vl_product.id	Produto
`movement_type`	texto(20)	sim		Tipo: IN, OUT, SALE, RETURN, ADJUST
`quantity`	inteiro	sim		Quantidade (positiva entra, negativa sai)
`stock_after`	inteiro	sim		Estoque depois do movimento
`reason`	texto(200)	não		Motivo
`order_id`	bigint	não	FK: vl_order.id	Pedido relacionado
`user_id`	bigint	não	FK: vl_user.id	Quem fez
`created_at`	data e hora	sim		Quando

Índices: `product_id`, `order_id`.

Capítulo 5

Como parear o banco

Três passos: subir o banco, criar o usuário e apontar a aplicação.

Sem configurar nada (H2)

Rode a aplicação. O banco é um arquivo em `backend/data/velo.mv.db`, criado na primeira execução com os dados de demonstração. Para começar do zero, pare a aplicação e apague a pasta `backend/data`.

1. Subir o banco

Com Docker, o arquivo `docker-compose.yml` já traz cada banco pronto, com usuário `velo` e banco `velo`:

```
docker compose --profile postgresql up -d
```

Perfil do Docker	Porta	Primeira subida
`postgresql`	5432	poucos segundos
`mysql`	3306	cerca de 20 segundos
`mariadb`	3307	cerca de 15 segundos
`sqlserver`	1433	cerca de 30 segundos; o serviço `sqlserver-init` cria o banco e o usuário e encerra sozinho
`oracle`	1521	de 1 a 2 minutos; acompanhe com `docker compose logs -f oracle` até aparecer `DATABASE IS READY TO USE`

Para desligar: `docker compose --profile postgresql down`. Acrescente `-v` para apagar também os dados.

2. Criar o banco e o usuário (sem Docker)

Troque `velo` pela senha que quiser.

PostgreSQL

```
CREATE USER velo WITH PASSWORD 'velo';
CREATE DATABASE velo OWNER velo ENCODING 'UTF8';
```

MySQL e MariaDB

```
CREATE DATABASE velo CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'velo'@'%' IDENTIFIED BY 'velo';
GRANT ALL PRIVILEGES ON velo.* TO 'velo'@'%';
FLUSH PRIVILEGES;
```

SQL Server

```
CREATE DATABASE velo COLLATE Latin1_General_100_CI_AS_SC_UTF8;
GO
CREATE LOGIN velo WITH PASSWORD = 'Velo@Senha2026';
GO
USE velo;
CREATE USER velo FOR LOGIN velo;
ALTER ROLE db_owner ADD MEMBER velo;
GO
```

Oracle

```
ALTER SESSION SET CONTAINER = FREEPDB1;
CREATE USER velo IDENTIFIED BY velo QUOTA UNLIMITED ON USERS;
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW TO velo;
```

3. Apontar a Velo para o banco

Linux ou macOS

```
export SPRING_PROFILES_ACTIVE=postgresql
export VELO_DB_URL=jdbc:postgresql://localhost:5432/velo
export VELO_DB_USER=velo
export VELO_DB_PASSWORD=velo
```

Windows (PowerShell)

```
$env:SPRING_PROFILES_ACTIVE = "postgresql"
$env:VELO_DB_URL = "jdbc:postgresql://localhost:5432/velo"
$env:VELO_DB_USER = "velo"
$env:VELO_DB_PASSWORD = "velo"
```

Os perfis já trazem o endereço padrão de cada banco, então `VELO_DB_URL` só é necessária se o banco estiver em outra máquina ou porta.

Perfil	URL padrão	Usuário / senha
`postgresql`	`jdbc:postgresql://localhost:5432/velo`	velo / velo
`mysql`	`jdbc:mysql://localhost:3306/velo?useUnicode=true&characterEncoding=utf8&serverTimezone=UTC`	velo / velo
`mariadb`	`jdbc:mariadb://localhost:3307/velo`	velo / velo
`sqlserver`	`jdbc:sqlserver://localhost:1433;databaseName=velo;encrypt=true;trustServerCertificate=true`	velo / Velo@Senha2026
`oracle`	`jdbc:oracle:thin:@//localhost:1521/FREEPDB1`	velo / velo

Na primeira conexão o Flyway cria as 13 tabelas e registra a versão em `flyway_schema_history`. Com o banco vazio, a Velo cadastra os dados de demonstração; para não criar esses dados, defina `VELO_SEED=false`.

Problemas comuns

Sintoma	O que fazer
`Connection refused`	O banco não está rodando ou a porta é outra. Confira com `docker compose ps` ou ajuste `VELO_DB_URL`.
`password authentication failed` / `Login failed`	Usuário ou senha diferentes dos informados em `VELO_DB_USER` e `VELO_DB_PASSWORD`.
`Validate failed: Migrations have failed validation`	O banco já tem outro esquema. Use um banco vazio só para a Velo.
SQL Server: `The server selected protocol version TLS10`	Atualize o SQL Server ou mantenha `trustServerCertificate=true` na URL.
Oracle: `ORA-01950: no privileges on tablespace`	Dê cota ao usuário: `ALTER USER velo QUOTA UNLIMITED ON USERS;`

Capítulo 6

Evolução do esquema

Os arquivos `V1__schema.sql` de cada banco são gerados por `scripts/schema/gerar_esquema.py`, a fonte única do modelo. Para mudar o esquema depois da primeira versão publicada:

1. Altere a entidade Java e o modelo em `gerar_esquema.py`.
2. Crie um `V2__descricao.sql` em cada pasta de `db/migration` com o `ALTER TABLE` correspondente. Nunca edite um `V1` já aplicado: o Flyway recusa scripts alterados.
3. Rode `python scripts/schema/gerar_esquema.py --somente-doc` para atualizar a documentação sem tocar no `V1`.

> **Validação na subida**
>
> A aplicação usa `ddl-auto=none`: quem cria e altera tabelas é sempre o Flyway. Nos testes, o Hibernate confere se as entidades batem exatamente com o esquema gerado.